内容导航

1.世界坐标系
2.相机坐标系
3.相机坐标到世界坐标的转换
4.相机内外参参数
5.深度计算
6.egosmith论文及代码梳理

1.世界坐标系

世界坐标系(Xw,Yw,Zw)是目标物体位置的参考系，根据运算方便自由设置原点位置，如可以位于相机镜头中心或机械手的中轴，或AGV(Automated Guided Vehicle)的几何中心点等。其主要作用有：
（1）定义物体的三维坐标。
（2）标定的时候根据原点确定标定物的位置。
（3）给出相机坐标系相对于世界坐标系的位置，从而求出两个或多个相机之间的坐标关系。

2.相机坐标系

相机坐标系（Xc，Yc，Zc）是相机在自己角度上的坐标系，原点在相机的光心上，Z轴与相机光轴平行，即相机的镜头拍摄方向。

3.相机坐标系与世界坐标系之间的转换

世界坐标（Xw，Yw，Zw）与相机坐标（Xc，Yc，Zc）同为三维坐标（右手系，三轴互相垂直），两个坐标系的关系为刚体变换（刚体变换：当物体不发生形变时，对一个几何物体作旋转，平移的运动）。可以先凭空想象下，有两个坐标系A与B，如何将A坐标系下的坐标转换到B坐标系表示，首先将A坐标系以原点为基准任意旋转，使其x轴，y轴，z轴与B坐标轴平行且同方向，接着平移AB坐标系原点的直线距离，就可以将A坐标系下的坐标转换到B坐标系，这个旋转与平移就是需要求得的两个三维坐标之间的关系。
世界坐标系→相机坐标系

$$\begin{Bmatrix} X_c \\ Y_c \\ Z_c \end{Bmatrix} = R \begin{Bmatrix} X_w \\ Y_w \\ Z_w \end{Bmatrix} + T$$

相机坐标系→世界坐标系

$$\begin{Bmatrix} X_w \\ Y_w \\ Z_w \end{Bmatrix} = R^T (\begin{Bmatrix} X_c \\ Y_c \\ Z_c \end{Bmatrix} - T)$$

其中旋转矩阵R表示世界坐标系到相机坐标系的旋转，可以看成空间坐标分别沿着X，Y，Z轴的三个旋转矩阵点乘得到的结果。T是世界原点在相机坐标系中的坐标
当绕Z轴旋转θ角度，新旧坐标的关系为：

$$\begin{cases} x = x'cos\theta - y'sin\theta \\ y = x'cos\theta + y'sin\theta \\ z = z' \end{cases}$$

用矩阵表示为

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} cos\theta & -sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = R_1 \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix}$$

同理，绕X轴，Y轴旋转φ和ω角度，可以得到：

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & cos\varphi & sin\varphi \\ 0 & -sin\varphi & cos\varphi \end{pmatrix} \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = R_2 \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} cos\omega & 0 & -sin\omega \\ 0 & 1 & 0 \\ sin\omega & 0 & cos\omega \end{pmatrix} \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = R_3 \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix}$$

于是，得到旋转矩阵R = R1xR2xR3，维度为3X3，T为平移矩阵，维度为3X1。

也可以表示为

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \begin{bmatrix} R_{cw} & t_{cw} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

4.相机内外参参数

4.1相机内参

内参参数包括焦距、偏心系数和倾斜系数。内参表达是相机坐标到图像坐标与像素坐标之间的转换关系。

1.焦距：焦距是镜头焦点与传感器之间的距离，它将RGB摄像机的3D点投影到摄像机中的2D图像平面上。

2.偏心系数：指相机组装过程中由于镜头元件错位引起的偏移。

3.当2D图像平面倾斜，即x轴和y轴之间的角度不是90度时，需要另一种转换，即从矩形平面到倾斜平面的转换（在图像到像素坐标转换之前）。假设x轴和y轴之间的角度为θ，则点从矩形平面到倾斜平面的转换可以如下进行。

4.2 相机外参

相机外参表达的是相机与世界坐标系之间的转换关系。通常由3x3旋转矩阵R(或四元数)及3x1平移向量t组成。

其中R表示相机的姿态，

t表示坐标变换中的平移部分，是世界坐标系原点在相机坐标系中的坐标。

四元数包含一个标量和一个三维向量的组合q=(w,x,y,z)，w是标量，与旋转角度有关，w=cosθ/2。x,y,z是向量，与旋转轴有关，k是单位旋转轴，(x,y,z)=ksinθ/2。所以一个单位的四元数本质上编码了“绕哪个轴转多少度”

1.单位旋转轴k

单位旋转轴k是一个三维单位向量(模长为1)，k=[kx,ky,kz]^T

旋转轴	表示	对应欧拉角
绕 x 轴	$[1, 0, 0]^T$	roll
绕 y 轴	$[0, 1, 0]^T$	pitch
绕 z 轴	$[0, 0, 1]^T$	yaw
任意轴	归一化后的方向向量	—

例如，要表示"绕 (1, 1, 0) 方向的轴旋转"，先归一化：

$$\mathbf{k} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.707 \\ 0.707 \\ 0 \end{bmatrix}$$

与其他表示的关系

1.**轴角（Axis-Angle）表示**：(\theta, \mathbf{k})

即一对"角度 + 单位轴向量"，共 4 个数但只有 3 个自由度（\mathbf{k} 的归一化约束占掉一个）。

2.**四元数**：

$$q = \left(\cos\frac{\theta}{2},\ k_x \sin\frac{\theta}{2},\ k_y \sin\frac{\theta}{2},\ k_z \sin\frac{\theta}{2} \right)$$

3.**旋转矩阵**（Rodrigues 公式，由 \theta 和 \mathbf{k} 构造）：

$$R = I + \sin\theta\,[\mathbf{k}]\times + (1 - \cos\theta)\,[\mathbf{k}]\times^2$$

$$\text{其中}[\mathbf{k}]\times\text{是}\mathbf{k}\text{的反对称矩阵：}\ [\mathbf{k}]\times = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix}$$

5.相机内外参标定方法

5.1内参标定方法

内参标定通常使用张正友标定法，就是常见的棋盘格标定。代码可参考https://github.com/leo038/hand_eye_calibrate/blob/main/hand_eye_calibrate.py 中的`camera_calibrate`函数。

5.2外参标定方法

外参标定的核心是：已知多个点分别在 相机坐标系下的坐标和在世界坐标系下的坐标， 求它们之间的映射关系。常用求解PnP 的方法，即已知多个点， 在像素坐标系的二维坐标， 和在世界坐标系的三维坐标，并且已知内参， 求解旋转平移矩阵。

cv2 中提供了外参标定方法， cv2.solvePnP 只需提供对应的点对即可。

```
import cv2
import numpy as np

# 定义已知的3D空间点和对应的2D图像点
```

```
objectPoints = np.array([[0, 0, 0], [1, 0, 0], [0, 1, 0], [0, 0, 1]], dtype=np.float32)
imagePoints = np.array([[10, 20], [30, 50], [20, 70], [50, 40]], dtype=np.float32)

# 定义相机的内参矩阵和畸变系数
cameraMatrix = np.array([[1000, 0, 320], [0, 1000, 240], [0, 0, 1]], dtype=np.float32)
distCoeffs = np.zeros((4, 1), dtype=np.float32)

# 使用solvePnP函数求解相机的位姿
ret, rvec, tvec = cv2.solvePnP(objectPoints, imagePoints, cameraMatrix, distCoeffs,
flags=cv2.SOLVEPNP_EPNP)

if ret:
    print("Rotation vector:\n", rvec)
    print("Translation vector:\n", tvec)
else:
    print("Failed to solve PnP.")

res, _ = cv2.Rodrigues(rvec)
print(f"旋转矩阵： {res}")
```

6.深度计算方法